

SkyRoute

Milestone 3 — Performance Evidence

Name: AlaguVishaalBalaji | Roll No: ZDA24B036 | June 2026

1. Project Summary

SkyRoute is a Track B AI Recommendation Engine that stores aviation data across six normalised PostgreSQL tables (city, airline, users, flight, flight_price, booking). The system uses collaborative filtering over the booking table to recommend flights to users based on the booking behaviour of similar users. This report covers query performance benchmarking before and after index creation, a stored procedure implementation, and a trigger for automatic seat updates.

2. Test Environment

Parameter	Value
Database Engine	PostgreSQL 18.4
Operating System	Windows 11
Dataset Size	2718 booking rows, 6 tables, 8462 total rows
Timing Method	EXPLAIN ANALYZE (actual time, averaged over 3 runs)
Index Types Used	B-Tree (default PostgreSQL index type)
Test Conditions	Same dataset, same machine, indexes dropped before BEFORE runs

3. Identified Slow Queries and Justification

Three queries were selected for benchmarking based on their role in the recommendation engine and the likelihood of improvement from indexing:

Q1 — User Revenue Aggregation: Joins booking to users and filters on booking_status = 'Confirmed', then groups by user. Without an index on booking_status or user_id, this requires a full sequential scan of all 2718 booking rows.

Q2 — Route Popularity (3-way Join): Joins booking to flight to city (twice) and filters on booking_status. This is the core of the collaborative filtering engine. Every recommendation query passes through this join path.

Q3 — Single User Booking Lookup: Filters booking on a specific user_id (WHERE user_id = 50). This is the highest-selectivity query — it returns only 8 rows out of 2718. Without an index, PostgreSQL scans the entire table and discards 2710 rows.

4. EXPLAIN ANALYZE — Before and After Indexes

4.1 Query 1 — User Revenue Aggregation

Metric	Before Index	After Index
Execution Time	1.039 ms	1.011 ms
Scan Type on booking	Sequential Scan	Sequential Scan
Rows Scanned	2718 (1243 removed by filter)	2718 (1243 removed by filter)
Join Method	Hash Join	Hash Join
Improvement	—	Minimal (2.7%)

Interpretation: PostgreSQL chose to keep the sequential scan even after indexing. This is correct planner behaviour — when a query must process more than roughly 10-15% of a table's rows (here, 54% of booking rows are Confirmed), a seq scan is faster than an index scan because it reads pages sequentially from disk. The query planner correctly determined that fetching 1475 rows via an index would cause more random I/O than a single sequential pass. This is a textbook example of why indexes do not always improve performance.

4.2 Query 2 — Route Popularity (3-way Join)

Metric	Before Index	After Index
Execution Time	2.067 ms	1.947 ms
Scan Type on booking	Sequential Scan	Sequential Scan
Scan Type on flight	Sequential Scan	Sequential Scan
Join Method	Hash Join (3 levels)	Hash Join (3 levels)
Improvement	—	5.8%

Interpretation: Similar to Q1, the planner chose sequential scans because the query processes most of the booking and flight tables. The 5.8% improvement comes from reduced planning overhead now that the planner has statistics on the index columns. At production scale (millions of rows), the `idx_flight_route` composite index would become critical — route lookups would switch from full scans to index range scans, with improvements of 10x or more.

4.3 Query 3 — Single User Booking Lookup (High Selectivity)

Metric	Before Index	After Index
Execution Time	0.742 ms	0.133 ms
Scan Type on booking	Sequential Scan	Index Scan (<code>idx_booking_user_id</code>)
Rows Scanned	2718 (2710 removed by filter)	8 (direct index lookup)
Buffers Used	shared hit=52	shared hit=29
Improvement	—	82% faster

Interpretation: This is the clearest demonstration of index benefit. Before the index, PostgreSQL scanned all 2718 rows and discarded 2710 (99.7% wasted work). After adding `idx_booking_user_id`, the planner switched to an Index Scan — it went directly to the 8 rows for `user_id = 50` without reading any other pages. Buffer usage also dropped from 52 shared hits to 29, confirming less data was read from memory. This query pattern (lookup by

user_id) is the most frequent operation in the recommendation engine, so this 82% improvement directly impacts real-world throughput.

5. Index Design Summary

Index	Type	Justification
idx_booking_user_id on booking(user_id)	B-Tree	Most frequent filter in recommendation engine. Proved 82% speedup on single-user lookup.
idx_flight_route on flight(origin_city_id, destination_city_id)	B-Tree (composite)	Supports collaborative filtering route grouping. Both columns needed together — composite avoids two separate index lookups.
idx_price_flight on flight_price(flight_id)	B-Tree	booking always joins to flight_price on flight_id. Eliminates seq scan on 5092 price rows.
idx_booking_flight_id on booking(flight_id)	B-Tree	Supports JOIN from booking to flight in route popularity queries.
idx_booking_status on booking(booking_status)	B-Tree	Every analytical query filters on booking_status. Low cardinality (4 values) — at scale this becomes a partial index candidate.

All indexes use the B-Tree type, which is PostgreSQL's default and is optimal for equality and range queries — exactly the patterns used in SkyRoute (WHERE user_id = X, WHERE booking_status = 'Confirmed', ORDER BY popularity DESC).

6. Stored Procedure

6.1 get_user_recommendations(p_user_id, p_limit)

A PL/pgSQL function that returns top-N personalised flight recommendations for a given user using collaborative filtering. It excludes routes the user has already flown and ranks remaining routes by how many other users have booked them. It also writes a log entry to recommendation_log on every call for audit purposes.

How to run:

```
SELECT * FROM get_user_recommendations(1, 5);
```

Output columns: flight_id, flight_number, origin, destination, airline_name, popularity_score

The function uses three CTEs internally: user_routes (routes already flown by the target user), route_scores (popularity count per route across all other users), and a final filter that excludes already-flown routes and returns the top N by score.

6.2 Trigger — trg_update_seats

An AFTER INSERT trigger on the booking table that automatically decrements seats_available in flight_price by 1 whenever a new booking is inserted. This keeps seat inventory consistent without requiring application-layer logic.

How it works:

```
-- fires automatically after every INSERT INTO booking
UPDATE flight_price SET seats_available = GREATEST(seats_available - 1, 0)
WHERE price_id = NEW.price_id;
```

The `GREATEST(..., 0)` guard prevents `seats_available` from going below zero if overbooking data is inserted.

7. Conclusions and Future Work

The benchmarking results show a clear pattern: indexes provide the most benefit on high-selectivity queries (Q3: 82% improvement) and diminishing returns on low-selectivity queries that process most of the table (Q1, Q2). This aligns with PostgreSQL's cost-based planner, which correctly chose sequential scans for bulk aggregations.

At the current dataset size (2718 rows), the absolute times are all under 3ms, so the practical impact is small. The real value of the indexes will emerge when the booking table grows to tens of thousands of rows — the `idx_booking_user_id` index will scale logarithmically while a sequential scan scales linearly.

Future optimisations to explore in the Final Submission:

- Partial index on `booking(user_id)` WHERE `booking_status <> 'Cancelled'` — would make the recommendation query even faster by excluding cancelled rows at the index level
- `VACUUM ANALYZE` after bulk loads — ensures the planner has up-to-date statistics
- Connection pooling via `pgBouncer` when the Flask API is deployed

8. How to Run `performance.sql`

```
psql -U postgres -d skyroute -f performance.sql
```

The script is structured in 7 sections: (1) drop indexes, (2) `BEFORE EXPLAIN ANALYZE`, (3) create indexes, (4) `AFTER EXPLAIN ANALYZE`, (5) create stored procedure and trigger, (6) create `recommendation_log` table, (7) test calls. Each section is labelled with comments.